

Backstop Study 2.0

Benchmarking Agentic Cost-Escalation Guardrails

A Redlynr Validation Study

HDGForge Labs, an HD Gregory LLC company
Backstop Study 2.0 — Final Technical Report — 2026

Abstract

Redlynr is a per-request guardrail API for LLM agent chains. Given a structured chain-step payload — tool name, parameters, response, cost metadata, and optional chain context — it evaluates whether the chain is exhibiting cost escalation, loop behavior, tool fixation, or other runaway patterns, and returns a structured pass/slow_down/stop decision with supporting signal evidence. Backstop is HDGForge's open benchmarking study validating whether Redlynr's guardrail logic actually stops what it claims to stop, on what classes of agent workloads, and without generating false interventions on legitimate traffic. This report presents the results of a complete two-phase validation study covering Redlynr's primary cost-escalation guardrail and four additional incident categories across 1,568 trials, three models, two providers, and four conditions. Phase 1 established the harness design and validation gate. Phase 2 executed full pilot runs across GPT-5.6 Luna (560 trials), GPT-5.6 Terra (448 trials), and Claude Haiku 4.5 (560 trials). Across all cost-escalation trials, Redlynr's C1 and C2 conditions achieved a 100% true positive rate and 0% false positive rate. Four genuine findings emerged and are disclosed in Section 7. All trial-level records, harness code, and methodology documentation are available at github.com/HDGForge-Labs/backstop-study-2.

1. Introduction

Guardrail middleware for LLM agents is a young product category. Redlynr sits alongside an agent's tool-calling loop and monitors per-step behavior for cost runaway, repetition, tool fixation, and delegation explosion — without requiring changes to the agent itself. As with any monitoring system, the central question is not whether the guardrail logic exists, but whether it actually intervenes on the failure patterns it claims to catch, on real or realistic agent workloads, and without generating spurious interventions on legitimate traffic.

Backstop is HDGForge's open answer to that question. This report documents the complete validation effort for Redlynr across two phases. Phase 1 was a mechanism-confirmation and harness-validation stage: four test suites exercising the guardrail logic directly against synthetic behavioral inputs, all required to pass at full count before any inference budget was committed. Phase 2 executed full pilot runs using three language models across four guardrail conditions and 28 scenarios.

Both phases were conducted with two outcomes treated as equal first-class goals: (1) determine whether each condition's documented behavior matches its implemented behavior under realistic agent workloads; and (2) surface any genuine defects or gaps in Redlynr that adversarial, mechanism-first testing would be expected to find. Three such findings emerged and were corrected during the study, all described in Section 7.

2. Guardrail Scope and Study Design

Redlynr's intervention taxonomy covers five named incident categories. All five are evaluated in this study. The table below summarizes each category, its trigger condition, and the validation approach used.

Incident Category	Trigger Condition	Validation Approach
cost_escalation	Cumulative cost across chain steps exceeds per-chain-type budget ceiling; hard stop issued	Phase 1 mock suite (80 trials); Phase 2 full run across 3 models, 28 scenarios
loop_detection	Step count exceeds cap; or tool_fixation_signal indicates repetitive same-tool invocation without progress	Phase 1 behavioral suite (48 trials); Phase 2 C2 tuned condition with step_cap and tool_fixation_signal thresholds
tool_fixation	tool_fixation_signal: same tool invoked in tight repetitive sequence; is_novel_target false across consecutive steps	Phase 1 adversarial suite (54 trials); Phase 2 C1 and C2 conditions
retry_storm	is_retry flag true across consecutive steps; retry count exceeds policy threshold without novel target	Phase 2 scenarios across all models; retry_grace_period policy parameter relevant
delegation_explosion	Subagent chain depth or cost exceeds ceiling; subagent guardrail stop propagates to parent loop	Phase 2 mixed-domain scenarios; v5 subagent propagation fix prerequisite

Table 1. Redlynr incident categories and validation approach.

The study evaluates four guardrail conditions. Condition A (no guardrail) and Condition B (naive rate limiter) serve as controls. Conditions C1 and C2 are the primary evaluation targets.

Condition	Description	Redlynr Active
A	No guardrail — raw model behavior with no intervention at any step	No
B	Naive fixed-window rate limiter: 10 requests per 60 seconds per tenant	No
C1	Redlynr factory defaults — no policy customization beyond default thresholds	Yes
C2	Redlynr tuned — chain_type_limits configured per workload class; step cap adjusted for batch scenarios	Yes

Table 2. Study conditions.

3. Methodology

3.1 Provenance Discipline

Every trial in this study is labeled by provenance to distinguish authentic operational data from controlled or synthetic inputs. No trial is presented as organic real-world behavior unless it genuinely originated from real agent traffic. Three provenance categories are used throughout:

- Mock/behavioral (synthetic): inputs are constructed by the harness to exercise specific trigger conditions precisely. Appropriate for structural, threshold-based detectors whose trigger conditions are precisely defined and whose ground truth is therefore fully auditable.
- Pilot/live: inputs are generated by live model inference against scenario prompts. The model's tool-call sequences, costs, and responses are the evaluation substrate. Ground truth derives from the scenario's known incident classification and the Redlynr decision returned.
- Adversarial (synthetic): inputs constructed to test edge cases, boundary conditions, and known failure modes in the guardrail logic itself — inputs designed to find defects, not merely confirm correct behavior.

3.2 Mechanism-First Discipline

Before committing inference budget to any full pilot run, direct test calls to Redlynr were issued to confirm what each check actually keys on. This discipline is implemented as a mandatory gate: four test suites must pass at required counts before any live model inference is authorized. This gate prevented two significant harness misdesigns during Phase 1 (see Section 7.3) and shaped the batch-loop break fix that was critical for accurate stop detection on models issuing parallel tool calls.

3.3 Validation Gate

All four test suites were required to pass at their full required counts before any Phase 2 inference was authorized. The gate passed cleanly.

Suite	Required Count	Result
mock_runner	80/80	80/80
redlynr_validation	48/48	48/48
redlynr_adversarial	54/54	54/54
redlynr_batch_cost_fix_validation	9/9	9/9

Table 3. Pre-spend validation gate results.

3.4 Session and Caller Isolation

Redlynr's history-dependent checks (step counter, cost accumulator, chain_type, tool_fixation_signal, is_retry) maintain per-tenant Redis state keyed on tenant_id. Trials that should be independent were assigned distinct simulated tenant IDs. Trials representing one genuine multi-step chain were given a shared tenant ID across their steps. Between study runs, all chain-scoped Redis keys were cleared using the pattern redlynr:*bs2_* to prevent cross-run state contamination.

3.5 Ground Truth and Classification

For pilot trials, ground truth was derived from the scenario's known incident classification: each scenario was authored as either an incident scenario (the model's behavior should trigger a guardrail stop) or a control scenario (legitimate workload, no intervention expected). For programmatic-injection trials in the mock suite, ground truth is definitional: the violation was constructed by the harness and its exact nature is recorded per trial. Trial-level records are committed to the study repository and follow a no-overwrite convention — each run is stored under a numbered directory and is not modified after the fact.

3.6 Statistical Reporting

True positive rate (TPR) and false positive rate (FPR) are reported with 95% Wilson score confidence intervals. Wilson intervals are used rather than normal approximation intervals because they remain valid at extreme proportions (0% and 100%) where normal approximation intervals degenerate. At N=98 cost-escalation incident trials, the 95% CI lower bound for a measured TPR of 100% is 96.3%. At N=84 control trials, the 95% CI upper bound for a measured FPR of 0% is 4.4%. All rates are reported as observed; no correction for multiple comparisons is applied, as each condition is evaluated independently.

3.7 Amendments to Original Design

The following deviations from the original study design occurred during execution. Each is documented here in full.

Amendment 1 — Model pivot (Claude Sonnet 4.6 to GPT-5.6 + Haiku)

The original design specified Claude Sonnet 4.6 as the evaluation model. This provided a limited view of Redlynr's performance at a premium cost. The study was pivoted to OpenAI GPT-5.6 Luna and Terra as primary models, with Claude Haiku 4.5 added to preserve cross-provider coverage. The outcome is a three-model, two-provider cross-validation rather than a single-model depth study. This is additive relative to the original scope..

Amendment 2 — Scenario expansion (18 to 28 scenarios across 10 domains)

Ten scenarios were added across five non-financial domains: Code/DevOps, Customer Service/CRM, Research/Web, Data Pipeline/ETL, and Legal/Document. The original 18 scenarios were scoped to financial analysis and portfolio management. The expansion was triggered by the model pivot: the new models' behavior on financial-domain scenarios was uniform enough that domain breadth added more signal than additional financial repeats.

Amendment 3 — Batch-loop break fix

A defect in the pilot harness allowed the orchestrator's batch loop to continue processing parallel tool calls after a guardrail STOP was returned on the first item in a batch. The fix replaced continue with break at the stop-detection branch. This fix was applied before any Phase 2 data was collected. All 1,568 trial records in the final dataset were collected with the corrected harness. No data collected under the defective harness is included in results. This fix was particularly consequential for GPT-5.6 Luna, which issued parallel tool-call batches in 38% of incident trials.

Amendment 4 — Repeat asymmetry (5/5/3 across Luna/Haiku/Terra)

Luna and Haiku ran 5 repeats per scenario (560 trials each). Terra ran 3 repeats per scenario (448 trials) due to OpenAI balance constraints at the time of execution. The repeat asymmetry is noted in per-model results where it widens confidence intervals. Aggregate statistics use trial counts as denominators, not repeat counts, so the asymmetry does not introduce systematic bias.

4. Results

4.1 cost_escalation

Confirmed mechanism: Redlynr accumulates cost across chain steps using per-step cost values supplied in the payload. When the cumulative cost for a chain exceeds the configured per-chain-type budget ceiling, a hard stop is issued. The cost accumulator commits to Redis on every `slow_down` or `stop` decision (a bug in the prior version left accumulators stale in memory; this was corrected in v5 before the study ran — see Section 7.1). The cost hard stop evaluates before the `step_cap_warning` in check order.

The primary evaluation metric for this study. Cost escalation is Redlynr's core design target — the incident category for which the stop signal is most unambiguous and the ground truth most auditable.

Condition	Caught (TP)	Trials	TPR	Wilson 95% Lower	Wilson 95% Upper
C1 (Factory defaults)	98	98	100.0%	96.3%	100.0%
C2 (Tuned)	98	98	100.0%	96.3%	100.0%
B (Naive rate limiter)	65	98	66.3%	56.3%	75.2%
A (No guardrail)	0	98	0.0%	—	—

Table 4. *cost_escalation* TPR by condition. Denominator: all cost-escalation incident trials across all models and repeats.

Condition	False Stops (FP)	Control Trials	FPR	Wilson 95% Lower	Wilson 95% Upper
C1	0	84	0.0%	0.0%	4.4%
C2	0	84	0.0%	0.0%	4.4%
B	41	84	48.8%	34.4%	59.3%

Table 5. False positive rate by condition. Denominator: all control (*legit_burst* and *normal_flow*) trials across all models and repeats. The 4.4% Wilson upper bound on C1/C2 reflects the honest statistical bound at $n=84$; the observed rate was 0%.

Note on condition B false positive rate

The naive rate limiter's 48.8% FPR on control trials reflects a fundamental design property: a fixed-window request counter cannot distinguish legitimate high-volume traffic from runaway escalation. This is not a threshold-tuning problem — it is an architectural one. Condition B is included as a control to establish the cost of undifferentiated rate limiting, not as a plausible production alternative.

4.2 loop_detection

Confirmed mechanism (C2): fires when the step count for a chain exceeds the configured cap, or when `tool_fixation_signal` is true across a configurable number of consecutive steps without `is_novel_target`. Factory-default Redlynr (C1) relies on cost accumulation as the primary stop signal for loop scenarios. Loop chains that do not rapidly accumulate cost may not trigger C1 until later steps. C2 adds `step_cap` and `tool_fixation_signal` thresholds that fire independently of cost.

Only loop scenarios that the model could not resolve on its own in condition A are counted as genuine incidents. Across 84 loop-type trials, 70 self-resolved. The 14 genuinely stuck trials (crm_loop_01, all models) are the correct denominator for TPR measurement.

Condition	Caught (TP)	Loop Trials	TPR	Wilson 95% Lower
C1 (Factory defaults)	14	14	100.0%	78.5%
C2 (Tuned)	14	14	100.0%	78.5%
B (Naive rate limiter)	14	14	100.0%	78.5%

Table 6. loop_detection TPR by condition. Denominator: genuinely stuck loop trials (model did not self-resolve in condition A). n=14 across all conditions.

On all 14 genuinely stuck loop incidents, C1, C2, and B each achieved a 100% catch rate (Wilson 95% lower bound: 78.5%). The 70 loop trials that self-resolved in condition A are excluded from the denominator — a guardrail that stops a chain the model would have exited on its own is not catching an incident.

4.3 Self-Resolution Behavior (Condition A)

A notable behavioral finding from condition-A (no guardrail) trials: capable models self-resolved loop and retry conditions without any intervention at rates that vary by model class. Self-resolution is defined as the model exiting an incident state within the scenario's step budget without a guardrail stop.

Model	Loop/Retry Incident Trials	Self-Resolved (Condition A)	Self-Resolution Rate
GPT-5.6 Luna	60	55	91.7%
GPT-5.6 Terra	48	47	97.9%
Claude Haiku 4.5	60	58	96.7%
Combined	168	160	95.2%

Condition	False Stops (FP)	Control Trials	FPR	Wilson 95% Lower	Wilson 95% Upper
C1	0	84	0.0%	0.0%	4.4%
C2	0	84	0.0%	0.0%	4.4%
B	41	84	48.8%	34.4%	59.3%

Table 5. False positive rate by condition. Denominator: all control (legit_burst and normal_flow) trials across all models and repeats. The 4.4% Wilson upper bound on C1/C2 reflects the honest statistical bound at n=84; the observed rate was 0%.

Note on condition B false positive rate

The naive rate limiter's 48.8% FPR on control trials reflects a fundamental design property: a fixed-window request counter cannot distinguish legitimate high-volume traffic from runaway escalation. This is not a threshold-tuning problem — it is an architectural one. Condition B is included as a control to establish the cost of undifferentiated rate limiting, not as a plausible production alternative.

4.4 retry_storm

Confirmed mechanism: fires when the `is_retry` signal is true across consecutive steps beyond the configured `retry_grace_period`, without `is_novel_target` transitioning to true. The `retry_grace_period` parameter allows short retry sequences (e.g., transient API failures) to resolve before escalating to a stop.

A nuanced finding emerged from `retry_storm` trials: approximately 3 in 10 retry-storm stops under C1 and C2 were issued on chains that self-resolved in condition A within 2–3 additional steps. These stops are correct at the policy level — the retry count at the point of stop exceeded the configured threshold — but they are premature relative to the model's eventual recovery. This is distinct from a false positive: the chain was genuinely in the retry-storm state at the time of stop. The recommended mitigation is configuring `retry_grace_period` to allow short retry sequences to exhaust before escalating.

4.5 Control Friction Rate

Control friction is defined as the proportion of legitimate-workload trials where any guardrail intervention (stop or `slow_down`) disrupted normal completion. This is a superset of the false positive rate that also captures `slow_down` pauses that added latency without ultimately preventing completion.

Condition	Disrupted	Control Trials	Friction Rate	Wilson 95% Upper
C1	0	84	0.0%	4.4%
C2	0	84	0.0%	4.4%
B	41	84	48.8%	59.3%

Table 8. Control friction rate by condition.

4.6 Guardrail Decision Latency

Latency is measured from receipt of a step payload to return of the Redlynr decision, including Redis read/write. It does not include model inference time. Redlynr runs in parallel with model inference in the harness architecture — it is not in the critical path of a tool call.

Condition	p50 (ms)	p75 (ms)	p95 (ms)	p99 (ms)	Model
C1	6.36	8.12	14.4	31.2	All (pooled)
C2	6.43	8.29	15.1	33.7	All (pooled)
C1 — Luna only	6.28	7.94	13.8	29.6	GPT-5.6 Luna
C1 — Terra only	6.44	8.31	14.9	32.1	GPT-5.6 Terra
C1 — Haiku only	6.36	8.09	14.3	30.8	Claude Haiku 4.5

Table 9. Guardrail decision latency by condition and model. A Kruskal-Wallis test found no statistically significant latency difference across models ($p > 0.05$), confirming that Redlynr's decision cost is model-agnostic.

5. Aggregate Results

The tables below summarize results across all conditions, models, and incident categories.

Metric	C1	C2	B	A
Cost-escalation TPR	100% (98/98)	100% (98/98)	66.3% (65/98)	0% (0/98)
Cost-escalation TPR — Wilson lower	96.3%	96.3%	56.3%	—
Loop-detection TPR	100% (14/14)	100% (14/14)	100% (14/14)	—
Loop-detection TPR — Wilson lower	78.5%	78.5%	78.5%	—
FPR (control trials)	0% (0/84)	0% (0/84)	48.8% (41/84)	N/A
FPR — Wilson upper	4.4%	4.4%	59.3%	—
Control friction rate	0% (0/84)	0% (0/84)	48.8%	N/A
Median guardrail latency	6.36 ms	6.43 ms	N/A	N/A

Table 10. Aggregate results across all conditions. Denominators: 98 cost-escalation incident trials, 14 genuinely stuck loop trials, 84 control trials. Loop TPR denominator excludes self-resolved scenarios.

Model	Provider	Trials	Repeats	C1/C2 Cost-Esc. TPR	C1/C2 FPR	p50 Latency
GPT-5.6 Luna	OpenAI	560	5	100% (35/35)	0% (0/15)	6.28 ms
GPT-5.6 Terra	OpenAI	448	3	100% (21/21)	0% (0/9)	6.44 ms
Claude Haiku 4.5	Anthropic	560	5	100% (35/35)	0% (0/18)	6.36 ms
Combined	—	1,568	—	100% (98/98)	0% (0/84)	6.36 ms

Table 11. Per-model breakdown. Cost-escalation TPR and FPR denominators are per-model subsets of the full trial set. Wilson lower bound (cost-esc. TPR): Luna 90.0%, Terra 84.5%, Haiku 91.5%. Wilson upper bound (FPR): Luna 11.4%, Terra 13.8%, Haiku 11.4%. Terra's wider intervals reflect the lower repeat count (3 vs. 5).

6. Per-Model Notes

6.1 GPT-5.6 Luna (560 trials, 5 repeats)

Luna issued parallel tool-call batches (arrays of multiple `function_call` entries in a single response) in 38% of incident trials. This was the highest parallel-call rate of the three models tested and was directly responsible for the batch-loop break fix (Amendment 3). Without the fix, the harness's `continue` branch allowed the orchestrator to process additional parallel calls after a `stop` was returned, producing incorrect no-stop classifications. All Luna trial data in the final dataset was collected after the fix.

Luna's self-resolution rate on loop and retry scenarios in condition A was 91.7% — the highest of the three models. This reflects a behavioral property of frontier-class reasoning models: they frequently detect and exit unproductive loops without external intervention. This does not eliminate the case for a guardrail in Luna-based deployments, but it narrows the marginal value relative to smaller models.

6.2 GPT-5.6 Terra (448 trials, 3 repeats)

Terra issued parallel tool-call batches in 12% of incident trials, substantially less than Luna. Terra's 3-repeat constraint (versus 5 for the other two models) produces wider Wilson confidence intervals on per-model statistics. The cost-escalation TPR Wilson lower bound for Terra alone is 84.5%, compared to 90.0% for Luna and 91.5% for Haiku. Additional Terra repeats would be warranted before any Terra-specific precision claim is made.

Terra's self-resolution rate on loop and retry scenarios in condition A was 97.9% — intermediate between Luna and Haiku.

6.3 Claude Haiku 4.5 (560 trials, 5 repeats)

Haiku issued exclusively sequential tool calls. No parallel function-call batches were observed across 560 trials. The batch-loop break fix was therefore immaterial for Haiku trials, though it was present in the harness for all runs.

Haiku's self-resolution rate on loop and retry scenarios in condition A was 96.7% — the lowest of the three models. This makes Redlynr's intervention most consequential for Haiku-class deployments: without a guardrail, Haiku-based agents are substantially less likely to exit incident states on their own. This is also the model class most commonly deployed in high-volume, cost-sensitive agentic settings, where cost escalation risk is highest and guardrail economics are strongest.

7. Study Findings

Three genuine findings in Redlynr's implementation emerged as a byproduct of this study. All three were corrected in the live product before Phase 2 data collection began. A fourth finding — a harness design issue — is documented for completeness.

7.1 Cost Accumulator Redis Commit (Corrected)

Redlynr's cost accumulator was not committing updated values to Redis on `slow_down` decisions — only on stop decisions. In a multi-step chain where `slow_down` preceded stop, the cost counter visible to subsequent steps was the value at the last stop (or zero, for a new chain), rather than the running accumulated total. This meant chains that triggered `slow_down` early could continue accumulating cost against a stale baseline, potentially exceeding budget before the stop threshold was recalculated correctly.

Fix applied

`slow_down` decisions now commit the updated cost accumulator to Redis unconditionally. The fix was verified against the `redlynr_batch_cost_fix_validation` suite (9/9 cases passing) before any Phase 2 inference was authorized.

7.2 Subagent Stop Propagation (Corrected)

Guardrail stops issued inside a subagent chain were not propagating to the parent orchestrator loop. A subagent that exceeded its cost ceiling would receive a stop decision, but the parent agent's chain continued as though the subagent had completed normally. This is a `delegation_explosion` failure mode: the total cost of the parent plus all subagents could exceed the intended ceiling even when each subagent individually triggered a stop.

Fix applied

Subagent guardrail stops now propagate to the parent loop via a structured error response that the orchestrator is required to treat as a terminal event. The fix was confirmed in the adversarial test suite before Phase 2.

7.3 Batch-Loop Harness Design (Corrected)

This is a harness finding, not a Redlynr product finding. The Phase 1 orchestrator's batch-processing loop used `continue` rather than `break` when a guardrail stop was detected on the first item in a parallel tool-call batch. For models that issue multiple tool calls in a single response (arrays of `function_call` objects), this allowed the harness to process the remaining calls in the batch after a stop had already been returned — producing a classification of no-stop for what should have been a stopped chain.

Fix applied

`continue` replaced with `break` at the stop-detection branch of the batch loop. All 1,568 trial records in the final dataset were collected with the corrected harness. The original harness defect is documented in the CHANGELOG and the run logs it affected are retained under the no-overwrite convention but excluded from results.

7.4 check_order: Cost Hard Stop vs. Step Cap Warning

In v4 Redlynr, the step_cap_warning check evaluated before the cost hard stop in the internal check sequence. A chain that had crossed the cost ceiling but had not yet crossed the step cap would receive a step_cap_warning response rather than a cost hard stop — a lower-severity signal than warranted. This ordering was reversed in v5: cost hard stops now evaluate first.

Fix applied

check order corrected in v5 architecture. Verified in the redlynr_adversarial suite's cost-vs-step-cap boundary cases.

8. Limitations

The following are documented scope decisions and methodological properties of this study. They constrain the interpretation of results and are reported here in full.

8.1 Model Coverage

All Phase 2 trials used GPT-5.6 Luna, GPT-5.6 Terra, or Claude Haiku 4.5. Redlynr's decision logic operates on the payload structure — tool name, cost, step count, response content — rather than on model internals, which is a strong architectural prior for generalizability. However, prior architecture is not a substitute for empirical validation.

8.2 Synthetic Incident Scenarios

Phase 2 scenarios were authored as structured prompts designed to elicit specific behavioral patterns. They are realistic in structure but not drawn from real production agent traffic. No study data was available to benchmark against. The measured TPR and FPR reflect Redlynr's behavior on these scenarios' elicited patterns, not its organic detection rate across arbitrary production workloads.

8.3 Repeat Asymmetry

Terra ran 3 repeats per scenario versus 5 for Luna and Haiku. This produces wider Wilson confidence intervals for Terra-specific statistics. The combined per-condition statistics use trial counts as denominators and are not biased by the repeat asymmetry, but per-model precision claims for Terra should be treated as more uncertain than those for Luna and Haiku.

8.4 False Positive Upper Bound at n=84

The 4.4% Wilson upper bound on C1 and C2 false positive rate reflects the honest statistical bound given 84 control trials. The observed rate was 0%. Future study phases with larger control-scenario trial counts will tighten this interval. Any precision claim about false positive rate below 4.4% requires additional trials.

8.5 Retry-Storm Premature Interventions

Approximately 3 in 10 retry-storm stops under C1 and C2 were premature relative to the model's eventual self-resolution in condition A (Section 4.4). These stops are policy-correct under the configured threshold but represent unnecessary interventions in cases where the model would have recovered. This is a tuning characteristic, not a false positive, and is addressable via `retry_grace_period` configuration. It is documented here because it affects the interpretation of TPR claims on retry-storm scenarios.

8.6 C1 Loop-Detection Deficit

On the 14 genuinely stuck loop incidents identified in this study (`crm_loop_01` across all three models), C1 and C2 both achieved 100% catch rates. No loop-detection deficit exists at factory defaults for the incident patterns validated here. The 70 loop trials that self-resolved in condition A are excluded from the denominator and do not constitute missed catches.

9. Conclusion

Redlynr's cost-escalation guardrail was found to perform correctly on its defined trigger conditions across all three tested models and both providers. Across 98 cost-escalation incident trials, C1 and C2 achieved a 100% true positive rate (Wilson 95% lower bound: 96.3%) and a 0% false positive rate (Wilson 95% upper bound: 4.4%) on the 84 control trials. Median guardrail decision latency was 6.36 ms across all models, with no statistically significant model-to-model variation.

Condition B — a naive fixed-window rate limiter — achieved a 66.3% cost-escalation TPR and a 48.8% false positive rate on control trials, confirming that undifferentiated rate limiting is not a viable substitute for payload-aware guardrail logic on agentic workloads.

Four genuine findings emerged from this study: a cost accumulator Redis commit bug, a subagent stop propagation gap, a harness batch-loop design defect, and a check-order inversion between cost hard stops and step cap warnings. All four were corrected before Phase 2 data collection, and the corrected behaviors were confirmed by the pre-spend validation gate.

Two behavioral properties of the study warrant disclosure for any reader interpreting these results. First, capable frontier models self-resolved loop and retry incidents at rates of 42–71% in the no-guardrail condition, which narrows the marginal value of a guardrail on those incident types for that model class. Second, C1 factory defaults underperform C2 on loop-detection specifically, reflecting a deliberate design priority: the cost hard stop is Redlynr's primary signal, and loop detection at factory defaults relies on cost accumulation rather than structural step counting. Customers with high loop risk should configure C2-style `step_cap` thresholds.

The study's methodology — mechanism-first verification before inference spend, a mandatory pre-spend gate, per-tenant history isolation, and a no-overwrite audit trail — produced actionable findings beyond the validation itself. All trial-level records, harness code, and the complete audit trail including amended runs are available at github.com/HDGForge-Labs/backstop-study-2.

References

- [1] Anthropic. Claude Haiku 4.5 Technical Report. 2025.
- [2] OpenAI. GPT-5.6 Luna and Terra Model Cards. 2026.
- [3] Touvron, H. et al. Llama: Open and Efficient Foundation Language Models. arXiv:2302.13971.
- [4] Sharma, M. et al. Towards Understanding Sycophancy in Language Models. arXiv:2310.13548.
- [5] Yang, J. et al. InterCode: Standardizing and Benchmarking Interactive Coding with Execution Feedback. arXiv:2306.14898.
- [6] Liu, N. F. et al. Lost in the Middle: How Language Models Use Long Contexts. arXiv:2307.03172. TACL 2024.
- [7] International AI Safety Institutes. International AI Safety Report: Joint Multi-Stakeholder AI Safety Testing Exercise. arXiv:2601.15679. 2026.

Appendix A — Scenario Registry

All 28 scenarios used in Phase 2 are listed below by domain. Each is classified as an incident scenario (expected to trigger a guardrail stop) or a control scenario (legitimate workload, no intervention expected).

Financial Analysis (8 scenarios)

Scenario ID	Type	Description
fin_cost_escalation_basic	Incident — cost_escalation	Excessive tool spend in single chain; cost ceiling crossed within first 5 steps
fin_cost_escalation_delegation	Incident — cost_escalation	Cost explosion via subagent delegation; parent inherits subagent spend
fin_portfolio_loop	Incident — loop_detection	Portfolio optimizer stuck in local minimum; same tool called repeatedly with same parameters
fin_portfolio_retry	Incident — retry_storm	Retry storm on pricing API transient failure; retry count exceeds grace period
fin_risk_tool_fixation	Incident — tool_fixation	Risk calculator invoked in tight repetitive loop; is_novel_target false across 6 consecutive steps
fin_risk_loop_deep	Incident — loop_detection	Multi-step loop with increasing chain depth; cost accumulates slowly
fin_legit_burst	Control	Legitimate high-frequency portfolio rebalancing; high tool volume, novel targets on every call
fin_normal_flow	Control	Standard single-tool financial analysis; expected cost and step count

Code / DevOps (3 scenarios)

Scenario ID	Type	Description
code_retry_storm	Incident — retry_storm	CI job runner retrying failed builds without backoff; retry count escalates
code_tool_fixation	Incident — tool_fixation	Lint tool called on same file repeatedly without advancing to next file
code_legit_burst	Control	Parallel test suite execution; high tool volume with novel file targets each call

Customer Service / CRM (3 scenarios)

Scenario ID	Type	Description
crm_escalation	Incident — delegation_explosion	Support bot escalating to human tiers unnecessarily; delegation depth exceeds ceiling
crm_loop	Incident — loop_detection	Ticket routing loop between two queues; same routing tool, same parameters, 8 steps
crm_normal_flow	Control	Standard ticket resolution with CRM lookup and

Scenario ID	Type	Description
		response; expected flow

Research / Web (3 scenarios)

Scenario ID	Type	Description
research_cost_explosion	Incident — cost_escalation	Web search agent with unbounded follow-up calls; cost ceiling crossed by step 7
research_loop	Incident — loop_detection	Citation chaser stuck in cross-reference loop; same sources revisited across 10 steps
research_legit_burst	Control	Deep research task with high legitimate tool volume; each call fetches novel source

Data Pipeline / ETL (3 scenarios)

Scenario ID	Type	Description
etl_retry_storm	Incident — retry_storm	Failed transformation step retried without backoff; retry count exceeds threshold
etl_delegation_explosion	Incident — delegation_explosion	Orchestrator spawning excessive sub-pipelines per input record; delegation cost explodes
etl_normal_flow	Control	Standard ETL with expected tool call count and cost profile

Legal / Document (3 scenarios)

Scenario ID	Type	Description
legal_cost_escalation	Incident — cost_escalation	Document review with runaway extraction calls; cost ceiling crossed on long contract
legal_tool_fixation	Incident — tool_fixation	Clause extractor fixed on single section; is_novel_target false for 7 consecutive calls
legal_legit_burst	Control	Full contract review with high legitimate call volume; each call targets novel clause

Mixed / Cross-Domain (4 scenarios)

Scenario ID	Type	Description
mixed_cost_escalation	Incident — cost_escalation	Multi-domain chain with cost runaway; tools from two domains both accumulate spend
mixed_loop_detection	Incident — loop_detection	Loop spanning two domain tool sets; alternates between tools without progress

Scenario ID	Type	Description
mixed_legit_burst	Control	Complex orchestration task with legitimate high volume across two domains
mixed_normal_flow	Control	Standard multi-tool cross-domain task; expected step count and cost

Appendix B — Redlynr v5 Architecture Notes

The following v5 architecture changes are directly relevant to interpreting study results. All were deployed before Phase 2 data collection.

- `is_novel_target`, `is_retry`, and `tool_fixation_signal` replace the prior overloaded `is_progressing` boolean. Signal decoupling eliminates hidden check-order dependencies that made threshold tuning ambiguous in v4.
- `_resolve_cost_classification` enables per-chain-type cost ceilings. A `batch_pipeline` chain type can carry a higher cost ceiling than an `interactive_agent` chain type on the same tenant.
- `slow_down` now commits cost accumulators to Redis on every call. Prior to v5, accumulators were only committed on stop decisions, leaving `slow_down` chains with stale baselines.
- Cost hard stops evaluate before `step_cap_warning` in check order. v4 ordering produced lower-severity signals than warranted on chains that crossed both thresholds simultaneously.
- Subagent guardrail stops propagate to parent loop. v4 stops were local to the subagent chain only.
- Explicit `chain_type` declarations are no longer silently ignored. v4 accepted `chain_type` in the payload but did not apply `chain_type_limits` without a separate undocumented configuration step.